Next, we set the parameter values used in the data generator to create the transactions. We set the minimum and maximum values for each `CustomerID`. We also create a dictionary of product types and set `min`, `max`, `mean`, `alpha`, and `beta` variables, which you use to generate random transaction amounts according to a distribution.

```
CustomerID_vars = {"min": 1234, "max": 1260}

Product_vars = {"None": {"min": 1000, "max": 25001, "mean": 15520, "alpha": 4, "beta": 10}}
```

Figure 2.21 – Dictionaries to hold variables for use within the define_specs function

With the variables set, we build out the functions to create the transaction data, starting with the `define_specs` function. The function accepts as input a product type (defined in the dictionary in *Figure 2.21*), a positive or negative label, and a timestamp; it returns a dollar amount for the transaction. *Figure 2.22* shows a portion of the code; the rest is in the accompanying notebook.

```
import dbldatagen as dg
from datetime import datetime
import dbldatagen.distributions as dist
from pyspark.sql.types import IntegerType, FloatType, StringType

def define_specs(Product, Label, currentTimestamp = datetime.now()):
  pVars = Product_vars[Product]
  if Product == "None":
    if Label:
      return (dg.DataGenerator(spark, name="syn_trans", rows=nRows, partitions=4)
        .withColumn("CustomerID", IntegerType(), nullable=False,
                    minValue=CustomerID_vars["min"], maxValue=CustomerID_vars["max"], random=True)
        .withColumn("TransactionTimestamp", "timestamp",
                    begin=currentTimestamp, end=currentTimestamp,nullable=False,
                   random=False)
        .withColumn("Amount", FloatType(),
                    minValue=pVars["min"],maxValue=pVars["max"],
                    distribution=dist.Beta(alpha=pVars["alpha"], beta=pVars["beta"]), random=True)
        .withColumn("Label", IntegerType(), minValue=1, maxValue=1)).build()
```

Figure 2.22 – Defining the define_specs function to generate transaction records

Next, we write a function to generate a single record by calling `define_specs` and including the current timestamp.

```
from pyspark.sql.functions import expr
from functools import reduce
import pyspark
import os

# Generate a record
def generateRecord(Product,Label):
  return (define_specs(Product=Product, Label=Label, currentTimestamp=datetime.now()))
```

Figure 2.23 – Defining a function to generate a single transaction record